



Auditing Report

Hardening Blockchain Security with Formal Methods

FOR



Boundless Transceiver



Veridise Inc.
Sep. 30, 2025

► **Prepared For:**

Wormhole
<https://wormhole.com/>

► **Prepared By:**

Tyler Diamond
Kostas Ferles

► **Contact Us:**

contact@veridise.com

► **Version History:**

Sep. 30, 2025	V2
Sep. 12, 2025	V1

Contents

Contents	iii
1 Executive Summary	1
2 Project Dashboard	3
3 Security Assessment Goals and Scope	4
3.1 Security Assessment Goals	4
3.2 Security Assessment Methodology & Scope	4
3.3 Classification of Vulnerabilities	5
4 Trust Model	6
4.1 Operational Assumptions	6
4.2 Privileged Roles	6
5 Vulnerability Report	8
5.1 Detailed Description of Issues	9
5.1.1 V-WMH-VUL-001: Unsafe downcasts	9
5.1.2 V-WMH-VUL-002: BlockRootOracle may accept commitments from the future	10
5.1.3 V-WMH-VUL-003: Issue from finding: Funds may be locked in BoundlessTransceiver	11
5.1.4 V-WMH-VUL-004: Maintainability Improvements	12
Glossary	13

From Sep. 8, 2025 to Sep. 9, 2025, Wormhole engaged Veridise to conduct a security assessment of their Boundless Transceiver. The security assessment covered [smart contracts](#) of Boundless Transceiver, which stores Ethereum beacon chain block roots and verifies that events were emitted on another blockchain. Compared to the previous version, which Veridise previously audited*, the new version introduces the BoundlessTransceiver, which utilizes [Wormhole's](#) architecture to consume proof of a [zkVM](#) execution attesting that an event occurred on another chain. Veridise conducted the assessment over 4 person-days, with 2 security analysts reviewing the project over 2 days on commit `fa9d4c6`. The review strategy involved a tool-assisted analysis of the program source code performed by Veridise security analysts as well as thorough code review.

Project Summary. The security assessment covered the BeaconEmitter, BlockRootOracle (formerly BoundlessReceiver), BoundlessTransceiver and their respective dependencies. These contracts enable a form of [Inter-Blockchain Communication](#). Additionally, the assessment covered a zkVM program which proves that a given message was emitted as a log by a contract in a specified block.

The zkVM program utilizes [Steel](#) to prove the existence of a log. The program proves that the contract address emitted a log with the given encoded message at the block represented by the commitment.

The BeaconEmitter will read Beacon chain block roots for a requested slot from the Beacon roots contract address (as specified in [EIP-4788](#)), and proceed to multicast the slot number along with the returned root to Wormhole.

BlockRootOracle is deployed on another chain and stores block roots of another connected chain. This contract considers two sources of truth for confirmed roots. The first is consuming Wormhole messages emitted by the BeaconEmitter of the source chain. The second source consists of [RiscZero](#) proofs. The corresponding zkVM [program](#) (separate from the one mentioned above) proves that the derivation of the proposed new block root is correct. Note that the security analysts assumed the zkVM implementation is correct and protects against known attacks on Ethereum's consensus algorithm, e.g., balance drift attacks.

The BlockRootOracle contract stores which methods have attested to the validity of every block root. Users of the BlockRootOracle contract can then query for the root for a specific slot and also check whether the returned root has been attested to by either or both validation methods. The contract also exposes the `validateCommitment()`, which returns true only if the provided Steel commitment has been attested to.

The BoundlessTransceiver contract implements a Transceiver as specified by Wormhole's [Native Token Transfers \(NTT\)](#). Transceivers are deployed on chains to implement custom logic for verifying the legitimacy of cross chain messages. Specifically, the BoundlessTransceiver implements the logic for sending and receiving messages:

* The previous audit report, if it is publicly available, can be found on Veridise's website at <https://veridise.com/audits/>

- ▶ `sendMessage()`: The contract simply emits a message (in NTT format) that broadcasts the Wormhole Chain ID[†] that this contract is deployed on.
- ▶ `receiveMessage()`: The contract receives a message by verifying the proof provided by the zkVM program that is in scope of this security review. This confirms that the input block commitment contains an event, which was emitted by function `sendMessage()` from a trusted address of the source chain. After this has been confirmed, the corresponding block root (Steel commitment) is verified to have been observed by the `BlockRootOracle`.

Code Assessment. The Boundless Transceiver developers provided the source code of the Boundless Transceiver contracts for the code review. The source code appears to be mostly original code written by the Boundless Transceiver developers. It contains some documentation in the form of READMEs and documentation comments on functions and storage variables. The analysts note that the documentation is significantly more comprehensive than in the previous review. To facilitate the Veridise security analysts' understanding of the code, the Boundless Transceiver developers provided the repository for the corresponding zkVM implementation.

The source code contained a test suite, which the Veridise security analysts noted contained both positive and negative test cases of the contracts. The analysts also noted that the `BeaconEmitterTest` mocked several of its dependencies.

Summary of Issues Detected. The security assessment uncovered 4 issues, 3 of which are assessed to be of low severity by the Veridise analysts. Specifically, the Veridise analysts uncovered an issue that might lock user funds in the protocol ([V-WMH-VUL-003](#)) as well as a few unsafe downcasts ([V-WMH-VUL-001](#)). The Veridise analysts also identified 1 warning finding. The Boundless Transceiver developers have been notified about these issues and indicated an intent to fix them.

Disclaimer. We hope that this report is informative but provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the system is secure in all dimensions. In no event shall Veridise or any of its employees be liable for any claim, damages or other liability, whether in an action of contract, tort or otherwise, arising from, out of or in connection with the results reported here.

[†] Note that [Wormhole Chain IDs](#) are NOT the same as EVM `chainIds`



Table 2.1: Application Summary.

Name	Version	Type	Platform
Boundless Transceiver	fa9d4c6	Solidity	Ethereum

Table 2.2: Engagement Summary.

Dates	Method	Consultants Engaged	Level of Effort
Sep. 8–Sep. 9, 2025	Manual & Tools	2	4 person-days

Table 2.3: Vulnerability Summary.

Name	Number	Acknowledged	Fixed
Critical-Severity Issues	0	0	0
High-Severity Issues	0	0	0
Medium-Severity Issues	0	0	0
Low-Severity Issues	3	3	3
Warning-Severity Issues	1	1	1
Informational-Severity Issues	0	0	0
TOTAL	4	4	4

Table 2.4: Category Breakdown.

Name	Number
Data Validation	3
Maintainability	1



3.1 Security Assessment Goals

The engagement was scoped to provide a security assessment of Boundless Transceiver's smart contracts and Steel-based zkVM program. During the assessment, the security analysts aimed to answer questions such as:

- ▶ Does the source code deviate from its intended specification?
- ▶ Is the protocol protected against common attacks in Ethereum's consensus algorithm (e.g., long-range attacks)?
- ▶ Are external dependencies (e.g., Wormhole SDK, RiscZero verifier) being used properly?
- ▶ Are any common smart contract vulnerabilities present in the codebase?
- ▶ Is the EVM modeled properly by the protocol?
- ▶ Does the BoundlessTransceiver ensure only valid messages are authenticated?
- ▶ Does the protocol handle user funds properly?

3.2 Security Assessment Methodology & Scope

Security Assessment Methodology. To address the questions above, the security assessment involved a combination of human experts and automated program analysis & testing tools. In particular, the security assessment was conducted with the aid of the following techniques:

- ▶ *Static analysis.* To identify potential common vulnerabilities, security analysts leveraged Veridise's custom smart contract analysis tool, Vanguard, which is designed to find instances of common smart contract vulnerabilities, such as reentrancy and uninitialized variables.

Scope. The scope of this security assessment is limited to the following files:

- ▶ src/
 - BeaconEmitter.sol
 - BlockRootOracle.sol
 - BoundlessTransceiver.sol
 - interfaces/ICommitmentValidator.sol
 - lib/Beacon.sol
- ▶ crates/
 - common/src/lib.rs
 - zkvm/src/lib.rs
 - zkvm/guest/src/main.rs

Methodology. Veridise security analysts reviewed the reports of previous audits for Boundless Transceiver, inspected the provided tests, and read the Boundless Transceiver documentation. They then began a review of the code assisted by static analyzers.

During the security assessment, the Veridise security analysts regularly communicated with the Boundless Transceiver developers to ask questions about the code.

3.3 Classification of Vulnerabilities

When Veridise security analysts discover a possible security vulnerability, they must estimate its severity by weighing its potential impact against the likelihood that a problem will arise.

The severity of a vulnerability is evaluated according to the Table 3.1.

Table 3.1: Severity Breakdown.

	Somewhat Bad	Bad	Very Bad	Protocol Breaking
Not Likely	Info	Warning	Low	Medium
Likely	Warning	Low	Medium	High
Very Likely	Low	Medium	High	Critical

The likelihood of a vulnerability is evaluated according to the Table 3.2.

Table 3.2: Likelihood Breakdown

Not Likely	A small set of users must make a specific mistake
Likely	Requires a complex series of steps by almost any user(s)
	- OR -
Very Likely	Requires a small set of users to perform an action
	Can be easily performed by almost anyone

The impact of a vulnerability is evaluated according to the Table 3.3:

Table 3.3: Impact Breakdown

Somewhat Bad	Inconveniences a small number of users and can be fixed by the user
Bad	Affects a large number of people and can be fixed by the user
	- OR -
Very Bad	Affects a very small number of people and requires aid to fix
	Affects a large number of people and requires aid to fix
Protocol Breaking	- OR -
	Disrupts the intended behavior of the protocol for a small group of users through no fault of their own
	Disrupts the intended behavior of the protocol for a large group of users through no fault of their own



4.1 Operational Assumptions

In addition to assuming that any out-of-scope components behave correctly, Veridise analysts assumed the following properties held when modeling security for Boundless Transceiver.

- ▶ The protocol's external dependencies, such as Wormohole's NTT Transceiver and the Steel library, are assumed to be functionally correct and free of security vulnerabilities.
- ▶ The protocol is aware and protects against any limitations of the zkVM guest program used to generate the proofs.
- ▶ The NTT Manager in charge of the BoundlessTransceiver is properly deployed and configured.
- ▶ Administrators behave properly.

4.2 Privileged Roles

Roles. This section describes in detail the specific roles present in the system, and the actions each role is trusted to perform. The roles are grouped based on two characteristics: privilege-level and time-sensitivity. *Highly-privileged* roles may have a critical impact on the protocol if compromised, while *limited-authority* roles have a negative, but manageable impact if compromised. Time-sensitive *emergency* roles may be required to perform actions quickly based on real-time monitoring, while *non-emergency* roles perform actions like deployments and configurations which can be planned several hours or days in advance.

During the review, Veridise analysts assumed that the role operators perform their responsibilities as intended. Protocol exploits relying on the below roles acting outside of their privileged scope are considered outside of scope.

- ▶ Highly-privileged, emergency roles:
 - `BlockRootOracle.ADMIN_ROLE` can manually transfer the protocol to a new state, update the image ID of the ZKVM application, and update the permissible timespan.
 - The owner of the BoundlessTransceiver, which is set to the owner of the corresponding NTTManager, sets the `AuthorizedSource` for receiving messages from chains. This contains:
 - * The contract address of the source chain's deployed BoundlessTransceiver
 - * The `BlockRootOracle` to validate block commitments from the source chain.
 - * The `imageID` for verifying proofs of the Steel-based zkVM application.
- ▶ Highly-privileged, non-emergency roles:
 - `BlockRootOracle.DEFAULT_ADMIN_ROLE` can grant roles (including `ADMIN_ROLE`) to users.
- ▶ Limited-authority, emergency roles:
 - The initial deployer of the BoundlessTransceiver can pause and unpause the contract.

Operational Recommendations. Highly-privileged, non-emergency operations should be operated by a multi-sig contract or decentralized governance system. These operations should be guarded by a timelock to ensure there is enough time for incident response. It is also recommended that `BlockRootOracle` inherits from `OpenZeppelin's AccessControlDefaultAdminRules`, which automatically introduces appropriate timelocks. Highly-privileged, emergency operations should be tested in example scenarios to ensure the role operators are available and ready to respond when necessary.

Full validation of operational security practices is beyond the scope of this review. Users of the protocol should ensure they are confident that the operators of privileged keys are following best practices such as:

- ▶ Never storing a protocol key in plaintext, on a regularly used phone, laptop, or device, or relying on a custom solution for key management.
- ▶ Using separate keys for each separate function.
- ▶ Storing multi-sig keys in a diverse set of key management software/hardware services and geographic locations.
- ▶ Enabling 2FA for key management accounts. SMS should *not* be used for 2FA, nor should any account which uses SMS for 2FA. Authentication apps or hardware are preferred.
- ▶ Validating that no party has control over multiple multi-sig keys.
- ▶ Performing regularly scheduled key rotations for high-frequency operations.
- ▶ Securely storing physical, non-digital backups for critical keys.
- ▶ Actively monitoring for unexpected invocation of critical operations and/or deployed attack contracts.
- ▶ Regularly drilling responses to situations requiring emergency response such as pausing/unpausing.

5

Vulnerability Report

This section presents the vulnerabilities found during the security assessment. For each issue found, the type of the issue, its severity, location in the code base, and its current status (i.e., acknowledged, fixed, etc.) is specified. Table 5.1 summarizes the issues discovered:

Table 5.1: Summary of Discovered Vulnerabilities.

ID	Description	Severity	Status
V-WMH-VUL-001	Unsafe downcasts	Low	Fixed
V-WMH-VUL-002	BlockRootOracle may accept commitments . . .	Low	Fixed
V-WMH-VUL-003	Issue from finding: Funds may be locked . . .	Low	Fixed
V-WMH-VUL-004	Maintainability Improvements	Warning	Fixed

5.1 Detailed Description of Issues

5.1.1 V-WMH-VUL-001: Unsafe downcasts

Severity	Low	Commit	fa9d4c6
Type	Data Validation	Status	Fixed
Location(s)	► crates/common/src/lib.rs:63-69		
Confirmed Fix At	https://github.com/boundless-xyz/boundless-transceiver/pull/5,22b53bc,d498216		

The security analysts identified two issues where bits of input are assumed to be empty:

1. In `BoundlessTransceiver.sol:toUint16()` the function will return the first 16 left-hand bits of the input. The rest of the bits are discarded without checking their value.
2. In `lib.rs:fromWormholeFormat()` the function takes in a 32-byte input, and proceeds to cast the last 20 bytes as an Ethereum Address. However, the first 12 bytes are assumed to be zero with no explicit checks. The [Wormhole SDK](#) performs the check that the upper bits are zero, and this implementation should too.

Impact Unintended behavior may occur if the assumptions made surrounding the discarded bits are kept in their current form.

Recommendation

1. Range-check the input length and confirm the discarded bits are zero.
2. Check that the upper 12 bytes are zero, as in the Wormhole SDK.

Developer Response The developers have implemented both recommendations.

5.1.2 V-WMH-VUL-002: BlockRootOracle may accept commitments from the future

Severity	Low	Commit	fa9d4c6
Type	Data Validation	Status	Fixed
Location(s)	src/BlockRootOracle.sol:123-136		
Confirmed Fix At	https://github.com/boundless-xyz/boundless-transceiver/pull/5,cc51c30		

In the previous review of the protocol, Veridise analysts noted that the BlockRootOracle did not properly protect against long-range attacks in the `transition()` function (dubbed V-WMH-VUL-003 in the previous report). The developers fixed the core issue that was reported, however they did not introduce a new check that requires the new state is not derived from a future block.

Impact Non-canonical commitments from the future may be posted to the BlockRootOracle if the zkVM does not provide sufficient protections.

Recommendation Check that the `journal.postState` cannot have occurred after the current `block.timestamp`.

Developer Response The developers have implemented a `_validPostState()` function that verifies the justified checkpoint has occurred in the past. Since the zkVM application verifies that the justified checkpoint is derived from the finalized checkpoint, then both checkpoints are validated to have occurred in the past. They then call this function on the `journal.postState` from `transition()`.

5.1.3 V-WMH-VUL-003: Issue from finding: Funds may be locked in BoundlessTransceiver

Severity	Low	Commit	fa9d4c6
Type	Data Validation	Status	Fixed
Location(s)	src/BoundlessTransceiver.sol:18-176		
Confirmed Fix At	https://github.com/boundless-xyz/boundless-transceiver/pull/5,c4e36ee		

The BoundlessTransceiver inherits from Wormhole's Transceiver. The parent exposes the external, payable `sendMessage()` function for sending messages. This function simply calls the `_sendMessage()` function which is implemented by BoundlessTransceiver. This function does not read the `msg.value`, and does not forward the funds elsewhere. Therefore, any funds sent via `sendMessage()` will be lost forever in the contract.

Impact Any funds sent along with `sendMessage()` will be lost.

Recommendation Implement a fund-recovery method, or revert when `msg.value` is non-zero.

Developer Response The developers now check in `_sendMessage()` that the `msg.value` is zero.

5.1.4 V-WMH-VUL-004: Maintainability Improvements

Severity	Warning	Commit	fa9d4c6
Type	Maintainability	Status	Fixed
Location(s)	► crates/zkvm/src/lib.rs		
Confirmed Fix At	https://github.com/boundless-xyz/boundless-transceiver/pull/5,0bc5746,a592655		

The security analysts found the following locations where the maintainability of the code could be improved:

1. `src/BoundlessTransceiver.sol`:
 - a) The documentation on the `authorizedSources` mapping should specify that more than just the contract for verifying Steel commitments is stored.
 - b) `receiveMessage()`: The source variable should be of a memory type instead of storage, since no changes will be made.
 - c) `setAuthorizedSource()`: An event should be emitted when the `AuthorizedSource` of a `chainId` is updated.
 - d) `_quoteDeliveryPrice()`: Set the return value to be configurable to avoid requiring a redeployment if fees are to ever be collected by relayers.
2. `src/BeaconEmitter.sol`:
 - a) `emitForSlot()`: As mentioned in the previous report, document that this function reverts when not provided the exact Wormhole fee.
3. `crates/zkvm/src/lib.rs`:
 - a) The `SendTransceiverMessageEmitter` contract definition provides the bytecode to the `solc!` macro so the test suite can deploy the contract. The compiler version and method of generating the bytecode should be documented so reviewers of the codebase can reproduce the bytecode to confirm the contract is properly tested against.
4. `test/BlockRootOracle.t.sol`: Rename the test contract to `BlockRootOracleTest`.

Impact The understanding of the code is reduced, and maintainability issues may unintentionally introduce bugs in the future.

Recommendation Implement the recommendations.

Developer Response

1. All 4 subitems are addressed.
2. The documentation is added, with a small typo.
3. A TODO has been added for documenting the bytecode compilation.
4. The contract has been renamed.

Inter-Blockchain Communication A protocol which relies on light clients to send and verify the receipt of inter-blockchain messages. See also <https://ibcprotocol.org>. ¹

RiscZero RiscZero enables developers to create zero-knowledge applications by compiling vanilla Rust code into the RISC-V instruction set for consumption in their **zkVM** implementation. See <https://risczero.com/>. ¹

smart contract A self-executing contract with the terms directly written into code. Hosted on a blockchain, it automatically enforces and executes the terms of an agreement between buyer and seller. Smart contracts are transparent, tamper-proof, and eliminate the need for intermediaries, making transactions more efficient and secure. ¹

zero-knowledge circuit A cryptographic construct that allows a prover to demonstrate to a verifier that a certain statement is true, without revealing any specific information about the statement itself. See https://en.wikipedia.org/wiki/Zero-knowledge_proof for more. ¹³

zkVM A general-purpose **zero-knowledge circuit** that implements proving the execution of a virtual machine. This enables general purpose programs to prove their execution to outside observers, without the manual constraint writing usually associated with zero-knowledge circuit development . ^{1, 13}